

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по домашней работе № 3

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

Тестирование API сервиса «Lecter's Recipes» средствами Postman

Вариант 5

Выполнил: студент группы БР1.1
Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

Реализовать тестирование API средствами Postman: собрать комплексный сценарий основного процесса приложения и написать тесты, проверяющие ответы сервера.

2. Проверяемый сценарий

В качестве основного процесса выбран путь нового пользователя от регистрации до собственной публикации:

- 1) регистрация нового кулинара;
- 2) вход в систему и получение JWT;
- 3) открытие личного кабинета;
- 4) просмотр витрины рецептов;
- 5) фильтрация выдачи (сложность, время приготовления, сортировка, ингредиент);
- 6) просмотр страницы рецепта из отфильтрованного списка;
- 7) лайк и сохранение рецепта в личный кабинет;
- 8) комментирование рецепта;
- 9) подписка на автора и просмотр ленты подписок;
- 10) создание и публикация собственного рецепта.

Сценарий дополнен негативными проверками: повторная регистрация того же email, некорректное тело запроса, неверный пароль, обращение без токена, запрос несуществующего рецепта, пустой комментарий, повторная подписка и попытка изменить чужой рецепт.

3. Организация коллекции

Коллекция состоит из 28 запросов, выполняемых строго по порядку. Данные передаются между запросами через переменные коллекции, поэтому прогон не требует ручной подготовки: скрипт предзапроса первого шага генерирует уникальные email и username по текущей отметке времени, а тесты сохраняют выданный токен и идентификаторы.

```
// Pre-request первого запроса: делаем прогон повторяемым
const suffix = Date.now().toString(36);
pm.collectionVariables.set('username', 'guest_' + suffix);
pm.collectionVariables.set('email', 'guest_' + suffix + '@lecters.recipes');
pm.collectionVariables.set('password', 'Chianti2026!');

// Tests: сохраняем выданный токен для последующих шагов
var json = pm.response.json();
pm.collectionVariables.set('accessToken', json.accessToken);
pm.collectionVariables.set('userId', json.user.id);
```

Листинг 1 — передача состояния между запросами коллекции

4. Примеры тестов

Тесты проверяют не только код ответа, но и содержательные свойства: структуру тела, отсутствие служебных полей и соблюдение бизнес-правил.

```
// структура ответа и безопасность
pm.test('статус 201', () => pm.response.to.have.status(201));
pm.test('пароль не возвращается наружу', () => {
  pm.expect(json.user).to.not.have.property('passwordHash');
  pm.expect(json.user).to.not.have.property('password');
});

// корректность фильтрации
pm.test('все рецепты со сложностью hard',
  () => data.forEach((r) => pm.expect(r.difficulty).to.eql('hard')));
pm.test('сортировка по возрастанию времени', () => {
  const times = data.map((r) => r.cookTimeMinutes);
  times.forEach((t, i) => { if (i) pm.expect(times[i - 1]).to.be.at.most(t); });
});

// бизнес-правило: повторный лайк не должен увеличивать счётчик
pm.test('счётчик не вырос повторно', () => pm.expect(pm.response.json().likesCount)
  .to.eql(Number(pm.collectionVariables.get('likesAfterLike'))));
```

Листинг 2 — примеры проверок из коллекции

5. Состав проверок

Таблица 1 — Состав тестовой коллекции

Группа	Запросы	Что проверяется
Служебные	1	доступность сервиса, время ответа
Регистрация и вход	5	выдача токенов, конфликт email, валидация полей, неверный пароль
Личный кабинет	2	отказ без токена, соответствие профиля регистрационным данным
Витрина и фильтрация	5	непустая выдача, отсутствие черновиков, работа фильтров и сортировки
Просмотр рецепта	2	наличие состава и шагов, нумерация шагов, ответ 404
Социальные функции	8	лайк и его идемпотентность, избранное, комментарии, подписки, лента
Публикация	4	создание черновика, публикация, права доступа, удаление

Группа	Запросы	Что проверяется
Итого	28 запросов	75 проверок

6. Результат прогона

Коллекция запускается как из интерфейса Postman, так и из командной строки через newman, что позволяет включить её в конвейер сборки.

```
$ newman run recipe-api.postman_collection.json -e local.postman_environment.json

→ 12. Просмотр страницы рецепта
GET http://localhost:3000/api/v1/recipes/1 [200 OK, 3.2kB, 11ms]
□ статус 200
□ идентификатор совпадает с запрошенным
□ есть состав
□ есть пошаговые инструкции
□ шаги пронумерованы по порядку
```

	executed	failed
requests	28	0
test-scripts	28	0
assertions	75	0

```
total run duration: 625ms · average response time: 12ms
```

Листинг 3 — результат прогона коллекции в newman

7. Найденные дефекты

Первый прогон завершился ошибками, которые оказались дефектами самой коллекции, а не приложения. Оба исправлены:

- **SyntaxError: Identifier ‘data’ has already been declared.** Песочница Postman выполняет тестовые скрипты одного прогона в общей области видимости, поэтому повторное объявление переменной через `const` во втором скрипте роняет его. Объявления заменены на `var`;
- **пустая выдача при фильтрации по ингредиенту.** В качестве фильтра был выбран ингредиент, отсутствующий в демонстрационных рецептах. Запрос переписан на ингредиент, который заведомо встречается в наборе данных, а проверка дополнена сравнением с общим числом опубликованных рецептов — так тест доказывает, что фильтр действительно сужает выдачу.

Кроме того, коллекция была проверена на микросервисной версии приложения (LP2): при указании адреса шлюза все 28 запросов и 75 проверок проходят без изменений в самой коллекции, что подтверждает сохранение

публичного контракта API.

8. Выводы

Собран сквозной сценарий основного процесса из 28 запросов с 75 проверками, покрывающий и позитивный путь пользователя, и типовые ошибочные ситуации. Тестирование подтвердило корректность API и одновременно выявило две ошибки в самих тестах, что показало ценность запуска коллекции из командной строки: в интерфейсе Postman оба дефекта могли остаться незамеченными при поштучном выполнении запросов.